



Natural Language Processing Practical Journal

Name - Siddharth Kutty

Seat No. - 44030725030

Submitted to - Mrs. Swati Maurya

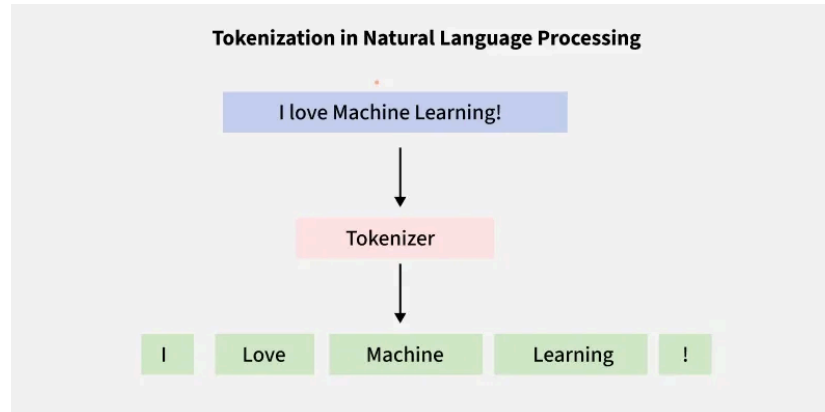
Index

Practical No.	Practical	Date
1	Apply Tokenization and find N-Gram model using NLTK and spaCy	
2	Identify base word using Stemming and Lemmatization, and find a Search Pattern	
3	Apply NER on given dataset and identify POS	
4	Naive-Bayes Theorem (without TF-IDF)	
5	Cocke-Younger-Kasami (CYK) Algorithm	
6	Maximum Likelihood Estimation	
7	Naive-Bayes Theorem (with TF-IDF)	
8	Multi-Word Expression (MWE) Detection	
9		
10		

Practical 1

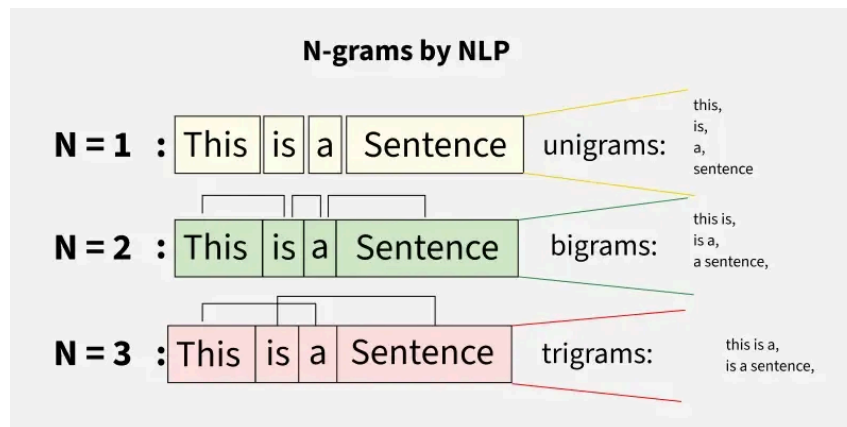
Tokenization

Tokenization is a fundamental step in Natural Language Processing (NLP). It involves dividing a Textual input into smaller units known as tokens. These tokens can be in the form of words, characters, sub-words, or sentences. It helps in improving interpretability of text by different models.



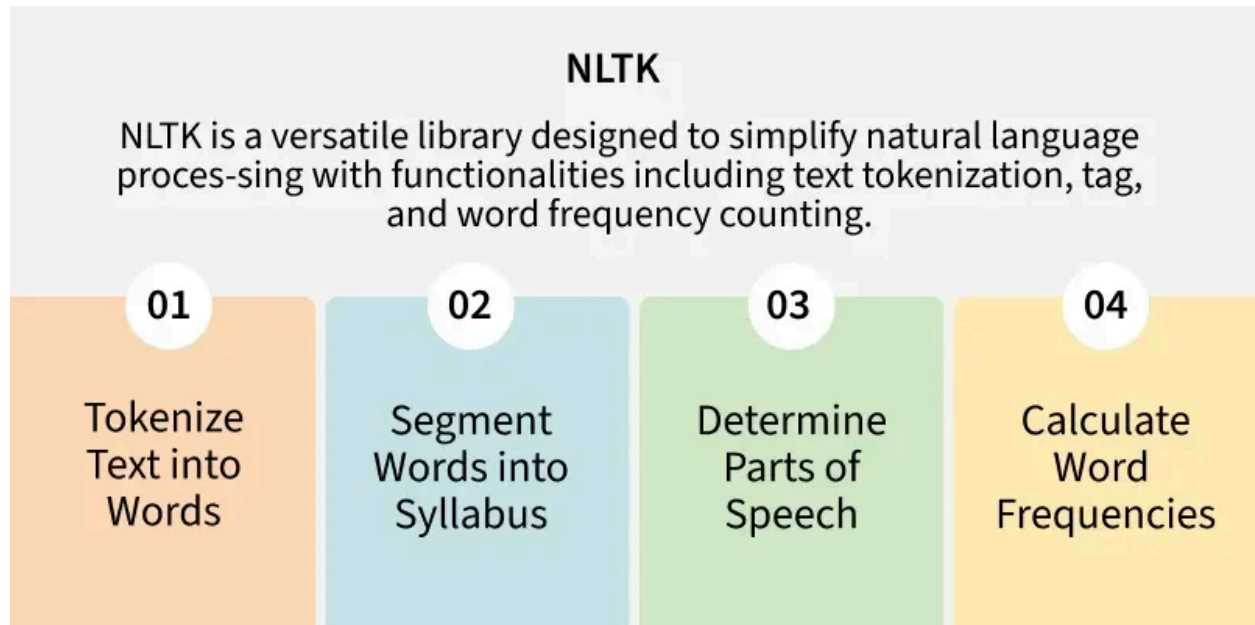
N Gram

N-gram is a contiguous sequence of 'N' items like words or characters from text or speech. The items can be letters, words or base pairs according to the application. The value of 'N' determines the order of the N-gram. They are fundamental concept used in various NLP tasks such as language modeling, text classification, machine translation and more.



NLTK

Natural Language Processing (NLP) plays an important role in enabling machines to understand and generate human language. Natural Language Toolkit (NLTK) stands out as one of the most widely used libraries. It provides a combination linguistic resources, including text processing libraries and pre-trained models, which makes it ideal for both academic research and practical applications.



spaCy

spaCy is a Python library used to process and analyze text efficiently for natural language processing tasks. It provides ready-to-use models and tools for working with linguistic data.

- Supports tokenization, POS tagging and dependency parsing
- Designed for speed and production use
- Works well with large text datasets
- Commonly used in NLP pipelines

Code -

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist
from nltk.util import ngrams

# Uncomment the following line to download the necessary resources for
tokenization
#nltk.download('punkt_tab')

# Sample text
text = "Natural Language Processing (NLP) is a field of artificial
intelligence that gives computers the ability to understand, interpret,
and generate human language."

print(f"Original Text:\n{text}")

# 1. Tokenization
# Breaking down the text into individual words or tokens
tokens = word_tokenize(text)
print("1. Tokenization (Word Tokens):")
print(tokens, "\n")

# 2. Frequency Distribution
# Counting the occurrences of each token
# You might want to filter out punctuation and convert to lowercase for
better frequency analysis
clean_tokens = [word.lower() for word in tokens if word.isalpha()]
fdist = FreqDist(clean_tokens)

print("2. Frequency Distribution (Top 10 most common words):")
for word, count in fdist.most_common(10):
    print(f"{word}: {count}")

# 3. N-gram Generation
# Generating sequences of N words (bigrams, trigrams, etc.)
# Bigrams (N=2)
bigrams_list = list(ngrams(clean_tokens, 2))
print("\n3. N-gram Generation ->")
```

```

print("Bigrams - top 10:")
print(bigrams_list[:10], "\n")

# Trigrams (N=3)
trigrams_list = list(ngrams(clean_tokens, 3))
print("Trigrams - top 10:")
print(trigrams_list[:10])

```

Output-

Original Text:

Natural Language Processing (NLP) is a field of artificial intelligence that gives computers the ability to understand, interpret, and generate human language.

1. Tokenization (Word Tokens):

['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'field', 'of', 'artificial', 'intelligence', 'that', 'gives', 'computers', 'the', 'ability', 'to', 'understand', ',', 'interpret', ',', 'and', 'generate', 'human', 'language', '.']

2. Frequency Distribution (Top 10 most common words):

language: 2

natural: 1

processing: 1

nlp: 1

is: 1

a: 1

field: 1

of: 1

artificial: 1

intelligence: 1

3. N-gram Generation ->

Bigrams - top 10:

[('natural', 'language'), ('language', 'processing'), ('processing', 'nlp'), ('nlp', 'is'), ('is', 'a'), ('a', 'field'), ('field', 'of'), ('of', 'artificial'), ('artificial', 'intelligence'), ('intelligence', 'that')]

Trigrams - top 10:

[('natural', 'language', 'processing'), ('language', 'processing', 'nlp'), ('processing', 'nlp', 'is'), ('nlp', 'is', 'a'), ('is', 'a', 'field'), ('a', 'field', 'of'), ('field', 'of', 'artificial'), ('of', 'artificial', 'intelligence'), ('artificial', 'intelligence', 'that'), ('intelligence', 'that', 'gives')]

Practical 2

Stemming

It is the process of producing morphological variants of a root/base word. Stemming programs are commonly referred to as stemming algorithms or stemmers. A stemming algorithm reduces the words “chocolates”, “chocolatey”, and “choco” to the root word, “chocolate” and “retrieval”, “retrieved”, “retrieves” reduce to the stem “retrieve”.

Lemmatization

Lemmatization is an important text pre-processing technique in Natural Language Processing (NLP) that reduces words to their base form known as a "lemma." For example, the lemma of "running" is "run" and "better" becomes "good."

Unlike stemming which simply removes prefixes or suffixes, it considers the word's meaning and part of speech (POS) and ensures that the base form is a valid word. This makes lemmatization more accurate as it avoids generating non-dictionary words.

Porter Stemmer

It is one of the most popular stemming methods proposed in 1980 by Martin Porter . It simplifies words by reducing them to their root forms, a process known as "stemming." For example, the words "running," "runner," and "ran" can all be reduced to their root form, "run." In this article we will explore more on the Porter Stemming technique and how to perform stemming in Python.

Code -

```
import nltk
from nltk.stem import PorterStemmer

# Uncomment the following line to download the necessary resources for
stemming
#nltk.download('wordnet')

pst = PorterStemmer()
words_to_stem = ["Giving", "Buying", "Studying"]
print("Stemming using Porter Stemmer:")
for i in words_to_stem:
    print(i + ": " + pst.stem(i))

from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
word_to_stem = ["cats", "cacti", "geese"]
print("\nLemmatization using WordNet Lemmatizer:")
for i in word_to_stem:
    print(i + ": " + lemmatizer.lemmatize(i))
```

Output-

Stemming using Porter Stemmer:

Giving: give

Buying: buy

Studying: studi

Lemmatization using WordNet Lemmatizer:

cats: cat

cacti: cactus

Practical 3

NER

Named Entity Recognition (NER) in NLP focuses on identifying and categorizing important information known as entities in text. These entities can be names of people, places, organizations, dates, etc. It helps in transforming unstructured text into structured information which helps in tasks like text summarization, knowledge graph creation and question answering.

POS

It is the process of labeling words within a text with their corresponding grammatical roles—such as noun, verb, adjective, or adverb—based on context and definition. It helps algorithms understand sentence structure and word meaning, crucial for tasks like machine translation, sentiment analysis, and information extraction.

Code -

```
import nltk
# Uncomment the following line to download the necessary resources for
stemming
#nltk.download('averaged_perceptron_tagger_eng')
#nltk.download('maxent_ne_chunker_tab')
#nltk.download('punkt_tab')
#nltk.download('words')
from nltk.tokenize import word_tokenize
from nltk import ne_chunk
string = "Siddharth Stays in India. He is working in Wipro Company"
str_tkn = word_tokenize(string)
str_tag = nltk.pos_tag(str_tkn)
NER = ne_chunk(str_tag)
print(NER)
```

Output -

(S
(PERSON Siddharth/NNP)
(ORGANIZATION Stays/NNP)
in/IN
(GPE India/NNP)
./.
He/PRP
is/VBZ
working/VBG
in/IN
(GPE Wipro/NNP)
Company/NNP)

Practical 4

Naive Bayes

The Naive Bayes algorithm is a probabilistic machine learning classifier widely used in Natural Language Processing (NLP) for tasks like spam detection and sentiment analysis. It is based on Bayes' Theorem, which calculates the probability of a category (e.g., "Spam") given a set of features (e.g., specific words in an email).

Code -

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# data
X = ["I love this", "This is bad", "Amazing product", "I hate it"]
y = ["Positive", "Negative", "Positive", "Negative"]

# model
v = CountVectorizer()
Xv = v.fit_transform(X)
m = MultinomialNB().fit(Xv, y)

# test
print(m.predict(v.transform(["I hate this product it is bad!!"])))
```

Output 1 -

for input "I hate this product it is bad!"
['Negative']

Output 2 -

for input "It is an amazing product"
['Positive']

Practical 5

Cocke-Younger-Kasami (CYK) Algorithm

It is used to solve the membership problem using a dynamic programming approach. The algorithm is based on the principle that the solution to problem $[i, j]$ can be constructed from solution to subproblem $[i, k]$ and solution to sub problem $[k, j]$. The algorithm requires the Grammar G to be in Chomsky Normal Form (CNF). Note that any Context-Free Grammar can be systematically converted to CNF. This restriction is employed so that each problem can only be divided into two subproblems and not more - to bound the time complexity.

Chomski Normal Form (CNF) Grammar

Chomsky Normal Form (CNF) is a restricted, simplified format for context-free grammars (CFGs) used to make parsing algorithms, like the CYK algorithm, more efficient. A grammar is in CNF if all production rules are in the form $A \rightarrow BC$ (two non-terminals) or $A \rightarrow a$ (one terminal).

Code -

```
def cyk(w, G):
    n = len(w)
    T = [[set() for _ in range(n)] for _ in range(n)]

    for i in range(n):
        for A, prods in G.items():
            if w[i] in prods:
                T[i][i].add(A)

    for l in range(2, n+1):
        for i in range(n-l+1):
            j = i + l - 1
            for k in range(i, j):
                for A, prods in G.items():
                    for p in prods:
                        if len(p) == 2 and p[0] in T[i][k] and p[1] in T[k+1][j]:
                            T[i][j].add(A)

    return 'S' in T[0][n-1]

G = {
```

```
'S': ['AB', 'BC'],  
'A': ['BA', 'a'],  
'B': ['CC', 'b'],  
'C': ['AB', 'a']  
}  
  
print(cyk("baaba", G))
```

Output -

True

Practical 6

Maximum Likelihood Estimation

Maximum Likelihood Estimation (MLE) is a statistical method for estimating the parameters of a probability distribution by maximizing a likelihood function. It finds the parameter values that make the observed data most probable, assuming a specific underlying distribution. The goal is to determine the best-fit parameters, such as mean or variance, that explain the collected data.

Code -

```
from collections import Counter

def mle(sentence):
    w = sentence.lower().split()
    n = len(w)

    uni = Counter(w)
    bi = Counter(zip(w[:-1], w[1:]))

    V = len(uni)

    uni_p = {word: (uni[word] + 1) / (n + V) for word in uni}
    bi_p = {(w1, w2): (bi[(w1, w2)] + 1) / (uni[w1] + V)
            for w1 in uni for w2 in uni}

    prob = uni_p[w[0]]
    for i in range(n - 1):
        prob *= bi_p[(w[i], w[i+1])]

    # Pretty output
    print("\nUnigram Probabilities:")
    for word in w:
        print(f"{word} : {uni_p[word]:.4f}")

    print("\nBigram Probabilities:")
    for i in range(n - 1):
        print(f"{w[i]} {w[i+1]} : {bi_p[(w[i], w[i+1])]:.4f}")

    print("\nSentence Probability:")
```

```
print(prob)

# Example
mle("I love NLP and I love coding")
```

Output -

Unigram Probabilities:

i : 0.2500
love : 0.2500
nlp : 0.1667
and : 0.1667
i : 0.2500
love : 0.2500
coding : 0.1667

Bigram Probabilities:

i love : 0.4286
love nlp : 0.2857
nlp and : 0.3333
and i : 0.3333
i love : 0.4286
love coding : 0.2857

Sentence Probability:

0.0004164931278633901

Practical 7

Term Frequency-Inverse Document Frequency

It is a numerical statistic used in Natural Language Processing (NLP) to reflect how important a word is to a document within a corpus. It boosts relevant words by multiplying high term frequency (within a document) by low document frequency (across the entire corpus), effectively filtering out common stop words.

Code -

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report

# Data
X = ["love movie", "amazing film", "hate it", "bad movie", "not amazing"]
y = [1, 0, 1, 0, 0]

# Vectorize
v = TfidfVectorizer()
Xv = v.fit_transform(X)

# Train
m = MultinomialNB().fit(Xv, y)

# Predict on training data (for demo)
y_pred = m.predict(Xv)

# Nice single prediction
test = "I love it"
test2 = "I hate it"
test3 = "It is not amazing"
pred = m.predict(v.transform([test]))[0]
print(f"'{test}' →", "Positive" if pred else "Negative")
pred = m.predict(v.transform([test2]))[0]
print(f"'{test2}' →", "Positive" if pred else "Negative")
pred = m.predict(v.transform([test3]))[0]
print(f"'{test3}' →", "Positive" if pred else "Negative")
```



```
# Evaluation metrics
print("\nMetrics:\n")
print(classification_report(y, y_pred))
```

Output -

'I love it' → Positive

'I hate it' → Positive

'It is not amazing' → Negative

Metrics:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	3
1	1.00	1.00	1.00	2
accuracy			1.00	5
macro avg	1.00	1.00	1.00	5
weighted avg	1.00	1.00	1.00	5

Practical 8

Multi-Word Expression (MWE) Detection

Multi-word expression (MWE) detection in NLP identifies groups of words (e.g., "break a leg," "machine learning") that behave as single lexical units with non-compositional meanings. It is crucial for improving machine translation, parsing, and sentiment analysis by treating these phrases as unified

Code -

```
import nltk
from nltk.collocations import BigramCollocationFinder
from nltk.metrics import BigramAssocMeasures
from nltk.corpus import stopwords

# Make sure to download stopwords if you haven't already
#nltk.download('stopwords')

corpus = [
    "I love machine learning",
    "Machine learning is fun",
    "I live in New York",
    "New York is a big city",
    "I like ice cream",
    "Ice cream is delicious"
]

stop = set(stopwords.words('english'))

# Combine all sentences into one list of words
words = [w for s in corpus for w in s.lower().split() if w not in stop]

# Find collocations
finder = BigramCollocationFinder.from_words(words)

# Keep only meaningful repeated pairs
finder.apply_freq_filter(2)

mwes = finder.nbest(BigramAssocMeasures.pmi, 10)
```

```
print("MWEs in corpus:\n")  
for m in mwes:  
    print(" ".join(m))
```

Output -

MWEs in corpus:

ice cream

machine learning

new york